

most of the modern UNIX derivatives, but it is not present in the traditional ones) and is pre-emptive. Thus you can pre-empt any task even if it is running in the kernel. You might have seen this in derivatives like IRIX and Solaris, but not in traditional ones.

The operating system supports an object-oriented device model and sysfs (user-space device filesystem). And you can also see that features like STREAMS are not available in Linux. Linux designers avoided many such poorly deployed ideas. What else? It is free and open source!

Tips

1. You can join the Linux kernel mailing list. To subscribe, go to <http://vger.kernel.org>. One thing that you need to take care of is that when you subscribe to lists like **linux-kernel**, **vger.kernel.org**, please create a new folder in your e-mail client (or create a filter, if you are in Gmail). Since this is a very high traffic list, you will get about 200 messages per day! By subscribing, you can keep yourself updated and interact with other developers.
2. I have already mentioned the ways to download the kernel source to your system (both by a direct download and command-line based). You can find the source under **/usr/src/your_linux_kernel_version**. You don't need to use this when you are performing some edits. You can maintain a directory under your **home** folder and do the trials. You need to switch to root (**su -**) only when you are about to install it. Also, when you move from one version to another you don't have to go for a full upgrade, but use patches. For an incremental patch, you can run the code below (after getting inside your source tree):

```
$ patch -p1 < ./patch-a.b.c
```

Normally, we apply this against the preceding version of the kernel that you want to use.

3. There are many utilities that support you while programming and debugging. There are even GUI versions available for many programs. The **Make config** (command line) utility allows you to do the modifications (customisation) to the kernel (please refer to Day 14). You can also try a **ncurses-based** graphical utility like **menuconfig** for this. **Make xconfig** (or try **gconfig** if you need a **gtk+** based one) is another X11-based graphical utility that aids you. You can make use of all these for customisation options.

By using **make defconfig** you can make a config based on the default setting that suits your architecture. This is a very good tool for beginners since you may not have configured the kernel earlier:

```
root@GNU-BOX:/usr/src/linux-source-2.6.27# make defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/basic/docproc
HOSTCC scripts/kconfig/conf
```

```
scripts/kconfig/conf.c: In function 'conf_askvalue':
scripts/kconfig/conf.c:104: warning: ignoring return value of 'fgets', declared with attribute warn_unused_result
scripts/kconfig/conf.c: In function 'conf_choose':
scripts/kconfig/conf.c:306: warning: ignoring return value of 'fgets', declared with attribute warn_unused_result
HOSTCC scripts/kconfig/lexgextext.o
SHIPPED scripts/kconfig/zconf.tab.c
SHIPPED scripts/kconfig/lex.zconf.c
SHIPPED scripts/kconfig/zconf.hash.c
HOSTCC scripts/kconfig/zconf.tab.o
In file included from scripts/kconfig/zconf.tab.c:2486:
scripts/kconfig/confdata.c: In function 'conf_write':
scripts/kconfig/confdata.c:501: warning: ignoring return value of 'fwrite', declared with attribute warn_unused_result
scripts/kconfig/confdata.c: In function 'conf_write_autoconf':
scripts/kconfig/confdata.c:739: warning: ignoring return value of 'fwrite', declared with attribute warn_unused_result
scripts/kconfig/confdata.c:740: warning: ignoring return value of 'fwrite', declared with attribute warn_unused_result
In file included from scripts/kconfig/zconf.tab.c:2487:
scripts/kconfig/expr.c: In function 'expr_print_file_helper':
scripts/kconfig/expr.c:1090: warning: ignoring return value of 'fwrite', declared with attribute warn_unused_result
HOSTLD scripts/kconfig/conf
*** Default configuration is based on 'U386_defconfig'
#
# configuration written to .config
#
```

After executing this, you can find the **.config** file in your source tree with entries like this:

```
# Automatically generated make config: don't edit
# Linux kernel version: 2.6.27.18
# Tue Aug 12 08:39:21 2009
#
# CONFIG_6BIT is not set
CONFIG_X86_32=y
# CONFIG_X86_64 is not set
CONFIG_X86-y
CONFIG_ARCH_DEFCONFIG="arch/x86/configs/U386_defconfig"
```

make oldconfig is another utility that helps you in validating and updating the configuration.

Please note that if you are on an earlier kernel, **make dep** may be required. You can see that I am now trying a more recent version. There are a few hurdles when you change the versions. I remember one reader had an issue when he used macros instead of syscall. For version 2.6.18 onwards, the API has been changed and you need to use syscall (...performs the system call whose assembly language interface has the specified number with the specified arguments.). More details are available at: www.kernel.org/doc/man-pages/online/pages/man2/syscall.2.html